

Demo Supplement: NL2TL-Tree

Natural Language to Nested Temporal-Logic Trees for
Robustness-Guided Flow-Matching Robot Planning

First Author, Second Author, Third Author et al.
Your Institution

June 5, 2026

Abstract

NL2TL-Tree is an interactive, *parameter-update-free* system that turns a plain-English instruction into a robot plan a non-expert can read and check. A frozen large language model parses the sentence into a *nested signal-temporal-logic (STL) tree*; the tree is rendered to a differentiable robustness objective that steers a frozen flow-matching trajectory planner (TeLoGraF); and a TurtleBot3 executes the plan in a Gazebo simulation, sensing obstacles online and re-planning to keep a non-negative keep-safe margin. Attendees type their own task and immediately see the parsed STL tree, the planned trajectory, and the robot carrying it out. Two self-contained front-ends share one map and task family: a lightweight 2D version (no robot middleware needed) and a full ROS2 + Gazebo version driving the simulated TurtleBot3.

1 Introduction

The NL2TL-Tree demonstration gives attendees a hands-on feel for how a natural sentence becomes a *checkable* robot plan with no training or fine-tuning. Through a simple text box they describe a task over a small map with three goal regions (A, B, C) in a field of obstacles, and the system shows each stage of the paper’s pipeline — language → nested STL tree → robustness objective → flow-matching plan → closed-loop execution. The emphasis is the *interpretable interface*: the nested STL tree is shown alongside the trajectory, so attendees see which part of their sentence produced which temporal constraint, and watch the robot satisfy it while avoiding obstacles it senses only at run time.

2 Attendee Interaction with the Demo

A presenter guides each interaction. The experience is built around two modules that share the same map and the same natural-language input box; they differ in what the attendee is invited to vary.

2.1 Module 1: From a typed sentence to an executed plan

The attendee types a task such as “Visit B, then A, then C in order, while always staying clear of every obstacle, and finish within 180 seconds.” On submission, the system (i) parses the sentence with the frozen LLM into a nested STL tree (e.g. a conjunction of three timed *finally* goals and a *globally*-not-unsafe keep-safe clause), (ii) grounds the region-named atoms (`reach_A`, ..., `unsafe`) to the fixed map coordinates, (iii) plans a trajectory with the frozen flow-matching planner, and (iv) drives the robot along it. Before anything moves, the attendee picks a coarse *environment-complexity* level and the system shows a *pre-flight completability* gauge — an estimated finish time (a quick collision-free path, scaled for the chosen clutter, plus the predicted planning time) checked against the deadline and graded *likely* / *tight* / *unlikely*. The gauge is *explanatory*: it reports the breakdown behind the verdict (movement time + predicted planning time vs. the deadline, under the chosen clutter), so when a task is tight or unlikely the attendee sees *why* — the tour is too long for the deadline, or planning latency eats the budget — and can revise the sentence (loosen the deadline, drop/reorder a region, relax the safety clause) and re-check instantly, closing a short feasibility-feedback loop before any motion (the feedback arrow in the paper’s pipeline figure). The parsed STL tree and the visit order are shown in the interface, and the robot is watched driving *live* — in RViz and Gazebo (ROS version) or in a live on-screen window (2D version), with the travelled path drawn in

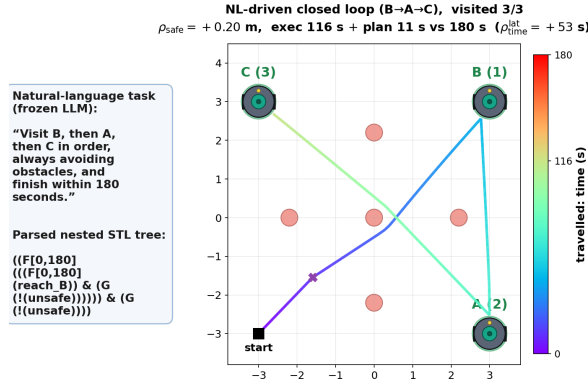


Figure 1: The 2D front-end. A typed sentence and its parsed nested STL tree are shown in the left panel; the targeted regions are highlighted in visit order and the time-coloured path is the executed closed-loop plan, with a TurtleBot3 marker on each reached region (the robot arrives *inside* the target; the per-leg path stops at the disk edge) and the keep-safe margin ρ_{safe} and the planning-latency-aware deadline (execution time + measured planner wall-clock vs. the deadline) in the title; the deadline is set to allow for the planner’s own latency, so the task stays in time once planning is counted. The ROS 2 + Gazebo front-end runs the identical map and task on a simulated TurtleBot3.

real time. Attendees are encouraged to change the regions, order, deadline, or safety clause and re-submit. Figure 1 shows the 2D front-end on the three-region B→A→C tour.

2.2 Module 2: Closed-loop sensing and keep-safe re-planning

The second module foregrounds the closed loop. The obstacles are *not* part of the specification: the robot senses them online (a 360° LiDAR in the Gazebo version; a sensing radius in the 2D version) and the keep-safe predicate $G(\neg \text{unsafe})$ is grounded by the sensed obstacle disks. When a newly sensed obstacle blocks the path, the planner re-plans from the current pose, and a runtime monitor triggers a stop-and-re-plan if the margin gets too small. Attendees watch the robot detour around obstacles it discovers en route; the demo reports the final keep-safe margin ρ over the executed trajectory and whether all requested regions were reached within the deadline. The demo also surfaces the planner’s *best-of-N self-check*: from the sampled prior’s reach-shortfall it flags out-of-distribution specifications and routes them to a classical A* fallback, so the attendee sees *which* engine produced the plan and why.

3 Technical Requirements

The following outlines the technical requirements for the NL2TL-Tree demo.

3.1 Equipment Provided by Presenters

The presenters provide a laptop (Linux, multi-core CPU, 16 GB RAM; no GPU required — the planner runs on CPU) and all cables. An optional external monitor improves visibility for onlookers.

3.2 Software and Data

The demo runs the open-source NL2TL-Tree code: a Python pipeline (parser, STL renderer, flow-matching planner) and, for the robot version, ROS 2 (Humble) + Gazebo Classic + the standard TurtleBot3 packages. The parser calls a hosted LLM (Groq); to stay robust to venue connectivity, a bundled offline parser produces the same nested STL tree for the example tasks, so the pipeline runs with no network. No private datasets are needed. The demo code (2D and ROS) and the experiment code are released at the repository linked from the paper.

3.3 Network Requirements

Internet access (Wi-Fi or wired) lets the live LLM parse handle arbitrary attendee sentences, but is *not* required: with the bundled offline parser the demo is fully functional offline.

3.4 Requirements from the Conference

We require a standard demo space with one table, two chairs (presenter + attendee), and a power connection. We bring all other equipment and anticipate no additional on-site support.